



Gruppo SkyNet
skynetunigroup@gmail.com

Norme di Progetto

Versione	<i>1.2.0</i>
Uso	<i>Interno</i>
Destinatari	<i>Prof. Tullio Vardanega, Prof. Riccardo Cardin</i>
Redattori	<i>Dario Pirrone, Andrea Fistarol, Edoardo Granziero, Sara Ristovic, Marco Barbiero</i>
Verifica	<i>Marco Barbiero, Dario Pirrone, Samuele Bertin, Edoardo Granziero</i>
Approvazione	<i>Samuele Bertin, Dario Pirrone, Andrea Fistarol, Edoardo Granziero</i>

Versioni del documento

Ver.	Data	Descrizione	Autore	Verificatore
1.2.0	2026/08/31	Aggiornamento della struttura della <i>repository</i> e dei <i>workflow</i>	Andrea Fistarol	Edoardo Granziero
1.1.0	2026/08/21	Ampliate e riviste le sezioni Codifica, Struttura Repository, Test di Unità e Test di Integrazione; creata la sezione Strumenti di Build	Andrea Fistarol	Marco Barbiero
1.0.1	2026/08/21	Aggiunta versione al riferimento del glossario e aggiunta data di accesso alle risorse web di riferimento	Marco Barbiero	Dario Pirrone
1.0.0	2026/08/13	Correzioni e verifica per revisione RTB	Andrea Fistarol	Edoardo Granziero
0.10.0	2026/08/07	Riscrittura della sezione 3.2	Marco Barbiero	Edoardo Granziero
0.9.0	2026/07/20	Riscrittura delle sottosezioni Verifica e Validazione	Sara Ristovic	Andrea Fistarol
0.8.0	2026/07/15	Riscrittura della sottosezione Documentazione	Sara Ristovic	Andrea Fistarol
0.7.1	2026/07/15	Implementazione delle correzioni della verifica dettagliata	Sara Ristovic	Edoardo Granziero
0.7.0	2026/07/14	Scrittura della sezione “Metriche per la Qualità” (5)	Dario Pirrone	Samuele Bertin
0.6.0	2026/07/07	Verifica dettagliata del documento intero	Sara Ristovic	—
0.6.0	2026/05/23	Scrittura della sottosezione “Sviluppo” (2.2)	Dario Pirrone	Samuele Bertin

Ver.	Data	Descrizione	Autore	Verificatore
0.5.0	2026/05/13	Scrittura della sezione “Processi di Supporto”	Edoardo Granziero	Dario Pirrone
0.4.0	2026/05/12	Scrittura della sottosezione “Fornitura”	Andrea Fistarol	Dario Pirrone
0.3.0	2026/04/27	Finita la scrittura della sezione “Processi Organizzativi”	Sara Ristovic	Marco Barbiero
0.2.0	2026/04/20	Scrittura delle sezioni “Gestione di Processi” e “Infrastruttura di Supporto”	Sara Ristovic	Marco Barbiero
0.1.0	2026/04/10	Strutturazione iniziale del documento; scrittura di “Introduzione”	Sara Ristovic	Marco Barbiero

Indice

1	Introduzione	6
1.1	Scopo del documento	6
1.2	Glossario	6
1.3	Riferimenti	6
1.3.1	Riferimenti Normativi	6
1.3.2	Riferimenti Informativi	7
2	Processi Primari	8
2.1	Fornitura	8
2.1.1	Avvio	8
2.1.2	Preparazione della risposta	8
2.1.3	Contrattazione	8
2.1.4	Pianificazione	8
2.1.5	Esecuzione e controllo	9
2.1.6	Revisione e valutazione	10
2.1.7	Consegna e completamento	10
2.2	Sviluppo	10
2.2.1	Analisi dei Requisiti	11
2.2.2	Progettazione	13
2.2.3	Codifica	14
3	Processi di Supporto	16
3.1	Documentazione	16
3.1.1	Attività previste	16
3.1.2	Progettazione	16
3.1.3	Produzione	17
3.1.4	Manutenzione	18
3.1.5	Documenti del progetto	18
3.1.6	Strumenti	18
3.2	Gestione della configurazione	19
3.2.1	Scopo	19
3.2.2	Aspettative	19
3.2.3	Descrizione	19
3.2.4	Versionamento	19
3.2.5	Sistemi Software Utilizzati	20
3.2.6	Struttura Repository	21
3.2.7	Strumenti di Build	24
3.3	Verifica	26
3.3.1	Verifica della documentazione	26
3.3.2	Analisi statica	26
3.3.3	Analisi dinamica	27
3.3.4	Test di Unità	27
3.3.5	Test di Integrazione	29
3.4	Validazione	30

3.4.1	Test di Sistema	30
3.4.2	Test di Accettazione	30
4	Processi Organizzativi	31
4.1	Gestione di Processi	31
4.1.1	Attività previste	31
4.1.2	Ruoli	31
4.1.3	Coordinamento	32
4.2	Infrastruttura di Supporto	34
4.2.1	Attività previste	34
4.3	Miglioramento di Processi	36
4.3.1	Attività previste	36
4.4	Formazione del Personale	37
4.4.1	Attività previste	37
5	Metriche per la Qualità	39
5.1	Classificazione, Nomenclatura e Struttura	39
5.2	Obiettivi di Qualità di Prodotto	39
5.2.1	Funzionalità	40
5.2.2	Affidabilità	40
5.2.3	Usabilità	40
5.2.4	Efficienza	40
5.2.5	Manutenibilità	41
5.2.6	Sicurezza	41
5.3	Metriche di Qualità di Processo	41
5.3.1	Processi primari	41
5.3.2	Processi di supporto	43
5.3.3	Processi organizzativi	44
5.4	Metriche di Qualità di Prodotto	44
5.4.1	Funzionalità	44
5.4.2	Affidabilità	45
5.4.3	Usabilità	46
5.4.4	Efficienza	47
5.4.5	Manutenibilità	48
5.4.6	Sicurezza	49

Elenco delle tabelle

1 Introduzione

1.1 Scopo del documento

Questo documento ha lo scopo di definire il *Way of Working_G* adottato dal gruppo per la durata del progetto **Code Guardian_G**. In particolare, esso definisce i processi, stabilisce le convenzioni e le *best practices_G*, e identifica gli strumenti da utilizzare specificando le relative modalità d'uso.

Le norme qui descritte seguono lo standard *ISO/IEC 12207:1995_G* suddividendo le attività in tre tipologie di processi:

primari	includono le attività fondamentali per la realizzazione del prodotto (sviluppo, fornitura, manutenzione, ecc.)
di supporto	assistono gli altri processi garantendo la qualità del software
organizzativi	aiutano il buon svolgimento del progetto. Stabiliscono l'infrastruttura e i processi di base, definiscono l'approccio al miglioramento continuo del <i>Way of Working</i> e contribuiscono a garantire un'adeguata formazione del personale.

Il documento mira a riflettere i principi dello standard adattandoli alle dimensioni del nostro gruppo e al dominio del nostro progetto. Ogni membro del gruppo è tenuto a seguire le regole stabilite in questo documento al fine di migliorare l'efficienza e l'efficacia del lavoro, garantendo così la qualità del prodotto finale. Poiché il gruppo mira al miglioramento continuo del proprio *Way of Working*, il documento verrà aggiornato incrementalmente durante l'intero periodo di lavoro e sarà completato solo alla fine del progetto.

1.2 Glossario

Termini tecnici e vocaboli particolarmente importanti il cui significato non sia immediatamente noto o chiaro vengono definiti nel documento **Glossario** che è in costante aggiornamento. Ogni termine presente nel **Glossario** è contrassegnato da una G a pedice ed è un collegamento ipertestuale che rimanda direttamente alla definizione corrispondente nel **Glossario**. Tali parole vengono contrassegnate solamente alla loro prima occorrenza in questo documento.

I termini evidenziati in grassetto identificano concetti tecnici ed elementi specifici del progetto, mentre i termini in corsivo indicano concetti tecnici di carattere generale, comuni al dominio dello sviluppo software.

1.3 Riferimenti

1.3.1 Riferimenti Normativi

- [Regolamento Progetto Didattico](#) (consultato il 13/08/2026)

1.3.2 Riferimenti Informativi

- [Glossario v1.0.0](#)
- [Standard ISO/IEC 12207:1995](#) (consultato il 13/08/2026)
- [Slide T02 del corso Ingegneria del Software \(Processi di Ciclo di Vita\)](#) (consultato il 13/08/2026)
- [Slide T03 del corso Ingegneria del Software \(Il Ciclo di Vita del SW\)](#) (consultato il 13/08/2026)
- [Slide T04 del corso Ingegneria del Software \(Gestione di Progetto\)](#) (consultato il 13/08/2026)
- [Slide T07 del corso Ingegneria del Software \(Qualità del Software\)](#) (consultato il 13/08/2026)
- [Slide T08 del corso Ingegneria del Software \(Qualità di Processo\)](#) (consultato il 13/08/2026)
- [Slide T09 del corso Ingegneria del Software \(Verifica e Validazione: introduzione\)](#) (consultato il 13/08/2026)
- [Slide T10 del corso Ingegneria del Software \(Verifica e Validazione: analisi statica\)](#) (consultato il 13/08/2026)
- [Slide T11 del corso Ingegneria del Software \(Verifica e Validazione: analisi dinamica\)](#) (consultato il 13/08/2026)

2 Processi Primari

Questa sezione definisce le attività fondamentali necessarie per l'intero ciclo di vita del software, includendo la fornitura del servizio e tutte le fasi tecniche dello sviluppo, dall'**Analisi dei Requisiti_G** alla codifica. Il loro scopo è fornire un percorso operativo strutturato per trasformare le necessità del **Proponente** in un prodotto software funzionante e conforme alle specifiche.

2.1 Fornitura

Questo processo comprende tutte le attività che vanno dalla preparazione della risposta al *capitolato_G* del **Proponente** fino alla consegna del prodotto e alla chiusura del progetto. Le attività sono trasversali e indipendenti dal tema specifico del progetto, poiché rappresentano le fasi in cui il progetto si colloca all'interno del processo di fornitura.

2.1.1 Avvio

Viene condotto un esame preliminare dei requisiti di ogni *capitolato* d'appalto tenendo conto delle politiche organizzative e di altri regolamenti, anche contattando i **Proponenti_G** per chiarimenti. I risultati sono documentati nella *Valutazione dei Capitolati* per l'invio ai **Committenti_G**.

Il gruppo decide collegialmente a quale *capitolato* candidarsi, assegnando a ogni *capitolato* un punteggio secondo parametri prestabiliti. Per ogni *capitolato*, deve essere data motivazione per l'accettazione o il rifiuto.

2.1.2 Preparazione della risposta

Il *capitolato* selezionato viene esplorato ulteriormente per raffinare la candidatura e massimizzare le possibilità di successo della candidatura. Va realizzato il *Preventivo_G* costi e assunzione impegni, secondo le modalità e i vincoli specificati nel Regolamento del progetto, e va preparata la Lettera di Presentazione.

2.1.3 Contrattazione

Essendo questo un progetto didattico, le modalità del contratto sono definite a priori dai **Committenti** nel Regolamento del progetto. Tuttavia, l'entità del prodotto da fornire è da definire con il **Proponente**, con il quale è necessario negoziare per adattare le richieste alle esigenze esterne del gruppo, in particolare gli altri obblighi universitari.

2.1.4 Pianificazione

Deve essere condotta una revisione delle richieste del *capitolato* per definire il *framework_G* del progetto e della qualità del prodotto SW. I risultati dell'**Analisi dei Requisiti** devono essere raccolti nel relativo documento e classificati. Gli obiettivi e i criteri di qualità individuati devono essere raccolti e organizzati nel *Piano di Qualifica_G*.

Deve essere scelto un modello di ciclo di vita. Il progetto è normato secondo un modello incrementale con metodologia *Agile_G* e *framework Scrumban_G*. La scelta di *Scrumban* è motivata dall'aver la struttura di *Scrum_G*, in grado di dare ordine al flusso di lavoro, ma con la flessibilità di *Kanban_G*, venendo così incontro alla ristrettezza di personale e all'inesperienza del gruppo nell'applicare *Scrum*, riducendo gli *overhead gestionali_G*. Si adottano *sprint_G*, periodi a durata fissa di sviluppo focalizzato, con una riunione all'apertura/chiusura per l'organizzazione del lavoro e la *retrospettiva_G* dello *sprint* precedente. I compiti individuati sono tracciati e assegnati nella board *Kanban* di *GitHub Projects_G*.

Devono essere impostati il **Piano di Progetto_G** (PdP) e il **Piano di Qualifica** (PdQ). Il **PdP** conterrà i costi totali previsti, in conformità con il *Preventivo Costi*, e dividerà il lavoro in *milestone_G* associate alle *baseline_G* stabilite dal **Committente**. A livello operativo, ogni *milestone* sarà raggiunta con più *sprint*, ognuno organizzato individualmente al suo inizio. Il **PdP** deve contenere *preventivi* dei costi di ogni *sprint* e monitorarne l'andamento tramite *Consuntivi_G* di fine *sprint*. Inoltre deve essere previsto il coinvolgimento del **Proponente**, con l'instaurazione di canali di comunicazione e riunioni.

Il PdQ definisce gli obiettivi di qualità necessari per garantire il successo dei processi e l'affidabilità del prodotto finale. In conformità con lo standard *ISO/IEC 12207:1995*, sono stati stabiliti obiettivi di qualità di prodotto e obiettivi di qualità di processo. Ogni obiettivo è monitorato attraverso metriche quantitative specifiche, per le quali sono stati fissati valori di accettazione e valori ottimali.

Code Guardian deve essere sviluppato internamente con l'uso dei servizi esterni previsti dal *capitolato*. I **Piani di Progetto** e **Qualifica** devono essere sviluppati in conformità con i requisiti qui sopra citati. Per una lista non esaustiva degli elementi da considerare nei piani, si può fare riferimento alla clausola 5.2.4.5 dello standard *ISO/IEC 12207:1995*.

2.1.5 Esecuzione e controllo

SkyNet deve implementare ed eseguire i piani di gestione del progetto sviluppati e deve sviluppare il prodotto secondo il Processo di Sviluppo (sezione 2.2), *sprint* dopo *sprint*.

Il progresso e la qualità del SW prodotto devono essere monitorati e controllati per tutta la durata del progetto. Questo compito deve essere continuativo, iterativo e deve provvedere a:

- Monitorare il progresso di performance tecniche, costi e programma e riportare lo stato del progetto.
- Identificare, registrare, analizzare e risolvere i problemi.

SkyNet deve interfacciarsi con le altre parti come specificato nel Regolamento del progetto e nei **Piani di Progetto**. Per quanto riguarda i **Committenti**, ciò significa partecipare ai **diari di bordo**, candidarsi alle revisioni previste e parteciparvi. Per quanto riguarda il **Proponente**, concordare i requisiti del progetto, cercare riscontro e conferma sui prodotti del progetto, partecipare alle riunioni esterne con l'azienda **proponente** e presentare e consegnare l'**MVP_G**.

2.1.6 Revisione e valutazione

Con l'azienda **Proponente Var Group** verranno svolte revisioni del progetto e del codice, oltre ad attività di Q&A in appuntamenti prefissati, in conformità con i processi di revisione e ispezione (*Audit_G*).

Con i **Committenti**, le revisioni **RTB_G** e **PB_G** avverranno nei momenti previsti dal *Regolamento del progetto* e quando il gruppo comunicherà di essere pronto. Verrà messo a disposizione l'intero *repository_G* con tutti gli artefatti, evidenziando i documenti essenziali.

Saranno eseguite attività di verifica e validazione, in conformità con i rispettivi processi qui definiti, per dimostrare che il prodotto software soddisfi pienamente i requisiti.

2.1.7 Consegna e completamento

SkyNet si impegna a consegnare il prodotto **Code Guardian**, composto dai seguenti elementi:

- **Analisi dei Requisiti**
- **Piano di Progetto**
- **Piano di Qualifica**
- **Glossario**
- **MVP funzionante**
- *Demo_G* live
- *Swagger_G API_G*
- Documentazione descrittiva del progetto
- *Bug_G Reporting*
- Codice sorgente su *GitHub_G*

Non è prevista alcuna attività di installazione, supporto o manutenzione del prodotto. Con la consegna e accettazione il contratto è da considerarsi terminato.

2.2 Sviluppo

Il processo di sviluppo regola l'organizzazione tecnica del team nella realizzazione del sistema. Per gestire in modo efficace la complessità del progetto e assegnare i compiti in modo delineato, le attività sono state suddivise in tre fasi principali:

- **Analisi dei Requisiti**
- **Progettazione**
- **Codifica**

Da questo processo ci si attende la realizzazione di un prodotto robusto e aderente alle richieste del **Proponente**. L'**Analisi dei Requisiti** permette di chiarire i bisogni del **Proponente** e di ridurre le ambiguità iniziali. La progettazione traduce tali requisiti in una soluzione tecnica coerente, mentre la codifica realizza concretamente il prodotto, producendo codice strutturato.

2.2.1 Analisi dei Requisiti

L'analisi ha l'obiettivo di formalizzare i bisogni del **Proponente** e tradurli in specifiche tecniche. Le finalità specifiche includono:

- definire con precisione i confini e gli obiettivi del software;
- individuare ed estrarre tutti i requisiti di sistema;
- modellare le funzionalità attese tramite *casi d'uso*_G;
- identificare gli *attori*_G che interagiranno con l'applicativo;
- fornire ai **Progettisti**_G una base di riferimento da rispettare durante lo sviluppo del sistema;
- preparare una base solida per le successive attività di verifica.

Questa attività, in carico principalmente al ruolo di **Analista**_G, porta alla stesura e all'aggiornamento del documento **Analisi dei Requisiti**. Tale documento costituisce la base di conoscenza fondamentale per lo sviluppo, mappando dettagliatamente il dominio del problema e le interazioni previste.

Casi d'uso

I *casi d'uso* descrivono le interazioni tra gli *attori* e il sistema, evidenziando gli obiettivi che l'utente intende raggiungere e il comportamento atteso dell'applicativo.

Per la loro classificazione viene utilizzata la seguente convenzione:

UC[Identificativo]: [Nome Descrittivo]

- **UC** (acronimo di *Use Case*);
- **Identificativo**: sequenza numerica progressiva che identifica in modo univoco il caso d'uso. Può essere composta da uno o più livelli separati da punti, in base al grado di dettaglio del caso d'uso;
- **Nome Descrittivo**: titolo sintetico che riassume l'interazione o l'obiettivo descritto dal caso d'uso.

Struttura

I *casì d'uso* devono esplicitare i seguenti elementi (tra parentesi tonde sono opzionali) e seguire la seguente struttura:

UC[Identificativo]: Titolo

Breve descrizione.

- **Attore Primario:**
- **(Attori Secondari):**
- **Trigger:**
- **Precondizioni:**
 - Condizione 1.
 - ...
- **Postcondizioni:**
 - Condizione 1.
 - ...
- **Scenario Principale:**
 1. Passo 1.
 2. Passo 2.
 3. ...
- **(Scenari alternativi):** breve descrizione degli eventuali flussi alternativi
- **(Specializzazioni):**
- **(Inclusioni):**
- **(Estensioni):**

Requisiti I requisiti identificano le singole funzionalità, i vincoli o le metriche di performance richieste al sistema. La loro tracciabilità è assicurata dal seguente formato:

$R[\text{Categoria}] . [\text{Identificativo}]$

- **R:** abbreviazione di Requisito;
- **Categoria:**
 - **F:** *Funzionale*, descrive un comportamento o un'operazione del sistema;
 - **Q:** *Qualitativo*, impone uno standard di sviluppo da rispettare;
 - **V:** *Vincolo*, indica una limitazione tecnologica o architetturale;
 - **S:** *Sicurezza*, definisce i requisiti legati alla sicurezza di SW.
- **Identificativo:** indice numerico progressivo per l'identificazione univoca.

2.2.2 Progettazione

L'attività di progettazione ha lo scopo di definire l'architettura logica e strutturale del sistema software, traducendo i requisiti emersi dall'analisi in soluzioni tecniche concrete. L'obiettivo principale è gestire la complessità del sistema attraverso scelte progettuali chiare e coerenti, favorendo la comprensibilità, la manutenibilità e la futura estendibilità del prodotto.

Le scelte progettuali vengono formalizzate graficamente e testualmente all'interno della documentazione tecnica, avvalendosi anche del linguaggio di modellazione *UML (Unified Modeling Language)_G*.

Suddivisione delle Fasi Progettuali Il ciclo di progettazione di **SkyNet** viene affrontato in modo incrementale, suddividendosi in due macro-fasi strategiche:

- **Requirements and Technology Baseline (RTB):** questa fase è fortemente orientata allo studio di fattibilità e alla riduzione dei rischi tecnologici. Il team valuta i *framework*, le librerie e gli strumenti candidati per lo sviluppo. L'attività principale consiste nella realizzazione di un **Proof of Concept (PoC)_G**, un prototipo software semplificato utile a verificare la fattibilità dell'idea di progetto e l'integrazione tra le tecnologie scelte, prima della produzione su larga scala.
- **Product Baseline (PB):** una volta consolidata la fattibilità tecnologica, si passa alla progettazione di dettaglio del sistema definitivo. In questa fase viene definita l'architettura logica completa, specificando l'organizzazione dei pacchetti, la struttura delle classi, le interfacce di comunicazione e le modalità di *deployment_G* dell'applicazione.

Tracciamento dei Componenti Al fine di garantire che nessuna funzionalità venga tralasciata e che non venga introdotto codice superfluo, **SkyNet** adotta una rigorosa strategia di tracciamento bidirezionale. Ogni componente software, classe o modulo descritto

nella progettazione deve essere esplicitamente associato a uno o più requisiti individuati durante l'analisi. Questo processo permette di verificare costantemente la copertura dei requisiti e facilita l'analisi dell'impatto delle modifiche durante l'evoluzione del sistema.

2.2.3 Codifica

La codifica è la fase in cui i **Programmatori_G** implementano le specifiche architetturali, rispettando rigorosamente le seguenti norme per garantire manutenibilità, leggibilità e qualità.

Lingua Inglese, inclusi nomi, commenti, *UI* e *logging*.

Convenzioni di Nomenclatura Tutti i nomi devono essere **descrittivi** dell'oggetto che rappresentano e **univoci** nel loro contesto.

TypeScript

- File: kebab-case
- Variabili e Funzioni: camelCase
- Classi e Tipi: PascalCase
- Interfacce: IPascalCase
- Costanti: UPPER_SNAKE_CASE

Python

- File, Variabili e Funzioni: snake_case
- Classi e Tipi: PascalCase
- Costanti: UPPER_SNAKE_CASE

Formattazione

- Indentazione: 2 spazi per *TS*, 4 spazi per *Python*.
- Lunghezza massima della riga: 100 caratteri.
- Virgolette: doppie per stringhe in *TS*, singole per *Python*.
- Punto e virgola a fine riga: sempre in *TS*, mai in *Python*.

Gestione Errori

- Eccezioni specifiche con messaggi chiari e descrittivi.
- *Logging*:
 - Livelli standard: DEBUG, INFO, WARNING, ERROR, CRITICAL.
 - Formato: [TIMESTAMP] [LEVEL] [MODULE] Message.

Commenti e Documentazione

- *Docstring* obbligatorie per tutte le funzioni o classi pubbliche, in formato *Google* per *Python* e *JSDoc* per *TS*.
- Commenti inline **solo** per logiche particolarmente complesse o non ovvie.

Struttura dei File

- Ordine: `import` → tipi → costanti → classi → funzioni.
- Una classe o funzione principale per file, raggruppando per responsabilità.
- La lunghezza del file non dovrebbe superare le 500 righe.

Strumenti

- *IDE*: *VS Code*, consigliato.
- *Linter*: *Biome* (*TS*), *Pylint* (*Python*).
- *Formatter*: *Biome* (*TS*), *Black* (*Python*).
- Analisi Statica: *SonarQube*.

3 Processi di Supporto

Questa sezione descrive le attività che assistono gli altri processi, con l'obiettivo di garantire la qualità, la coerenza e il buon esito del progetto. Tali processi includono la gestione sistematica della documentazione, il controllo delle configurazioni e le procedure di verifica e validazione necessarie per assicurare che ogni artefatto rispetti gli standard stabiliti.

3.1 Documentazione

Questo processo riguarda la registrazione delle informazioni prodotte durante un processo di ciclo di vita o durante un'attività di progetto. È fondamentale per molte ragioni:

- favorisce la chiarezza delle informazioni stabilite e riduce le ambiguità
- fornisce un accesso agevole alle informazioni da parte dei soggetti interessati
- favorisce una comprensione condivisa tra tutti i membri del gruppo
- definisce le regole e i processi da seguire e le metriche da rispettare
- descrive il prodotto software, spiegandone e facilitandone l'utilizzo.

Considerati gli scopi della documentazione, i documenti prodotti si rivolgono a diversi soggetti interessati, tra cui sviluppatori, responsabili di progetto, utenti finali del prodotto software e altri destinatari coinvolti nel progetto.

3.1.1 Attività previste

Le attività di documentazione previste per il progetto **Code Guardian** sono:

- **Progettazione** - in questa attività si eseguono:
 - l'identificazione dei documenti necessari
 - l'identificazione degli elementi principali comuni a tutti i documenti
 - la progettazione degli indici dei documenti
 - la definizione di uno o più *template_G* documentali da utilizzare durante la redazione dei documenti, al fine di standardizzarli.
- **Produzione** - la redazione dei documenti e la loro pubblicazione
- **Manutenzione** - l'aggiornamento dei documenti già presenti, al fine di preservarne la rilevanza e l'utilità.

3.1.2 Progettazione

Per ogni *milestone* principale del progetto (candidatura al *capitolato*, **RTB**, **PB**) devono essere identificati i documenti necessari per completarla con successo.

Gli elementi comuni alla maggior parte dei documenti sono:

- titolo
- logo del gruppo
- intestazione
- piè di pagina
- tabella delle informazioni principali sul documento:
 - versione
 - uso del documento (interno o esterno)
 - destinatari
 - redattori, ovvero i membri del gruppo che hanno contribuito alla stesura del contenuto
 - verificatori, ovvero i membri che hanno verificato il documento
 - approvazione, ovvero i membri che hanno approvato il documento.
- registro delle modifiche del documento, in cui ogni riga contiene:
 - versione
 - data di completamento della modifica
 - descrizione sintetica delle modifiche apportate
 - autore delle modifiche
 - verificatore delle modifiche
- indice
- contenuto del documento.

Nel *repository Git* di progetto è disponibile un *template* generico, utilizzabile come punto di partenza per la redazione di nuovi documenti. Il *template* definisce, posiziona e formatta gli elementi elencati sopra, con l'obiettivo di produrre una documentazione facilmente navigabile, professionale e standardizzata.

3.1.3 Produzione

Su ogni documento possono lavorare più membri contemporaneamente. Per ridurre il numero di *conflicti merge* da risolvere, ogni membro dovrebbe effettuare un *commit_G* nel *repository* del progetto al completamento di ogni piccola unità di lavoro.

Durante la redazione dei principali documenti di progetto, i termini che identificano concetti tecnici ed elementi specifici del progetto vanno evidenziati in grassetto, mentre i termini che indicano concetti tecnici di carattere generale, comuni al dominio dello sviluppo software, vanno evidenziati in corsivo.

I documenti vengono pubblicati sul [sito](#) del gruppo. Non tutti i documenti ci sono pubblicati, ma solo quelli relativi alla milestone principale corrente (candidatura al capitolato, RTB, PB). Tutti i documenti sono comunque presenti nel *repository Git* del progetto. Le versioni pubblicate sul sito vengono aggiornate quando il gruppo lo ritiene opportuno.

3.1.4 Manutenzione

Per tracciare chiaramente le modifiche apportate ai documenti, ogni documento contiene un registro delle modifiche, descritto nella sezione *3.1.2 Progettazione*.

Una parte importante della manutenzione di un documento consiste nella sua verifica generale. Questo compito viene svolto quando il gruppo lo ritiene opportuno, soprattutto dopo un lungo periodo di redazione. La verifica generale viene riportata nel registro delle modifiche del documento.

3.1.5 Documenti del progetto

I principali documenti prodotti e mantenuti dal gruppo sono:

- **Analisi dei Requisiti (AdR)** - raccoglie i *casi d'uso* e i requisiti del progetto, concordati con il **Proponente**
- **Piano di Progetto (PdP)** - contiene l'analisi dei rischi, la pianificazione del progetto e il monitoraggio del suo andamento e dei suoi costi
- **Norme di Progetto (NdP)** - definisce il *Way of Working* adottato dal gruppo
- **Glossario** - descrive i termini tecnici presenti nel resto della documentazione
- **Piano di Qualifica (PdQ)** - definisce gli obiettivi di qualità, le metriche e le attività di verifica e validazione necessarie per assicurare la qualità del prodotto e dei processi
- **Lettere di presentazione** - documenti redatti in occasione degli eventi principali dell'insegnamento (candidatura al *capitolato*, **RTB**, **PB**). Tali lettere non sono soggette a manutenzione successiva dopo il loro invio al **Committente** via email.

Gli scopi specifici dei documenti sono descritti all'interno dei documenti stessi.

3.1.6 Strumenti

Gli strumenti utilizzati in questo processo sono:

- *Overleaf_G* - editor $\text{\LaTeX}$ collaborativo basato su *cloud_G*, utilizzato per la redazione, la formattazione e l'affinamento finale dei documenti

- *TeXPage* - editor L^AT_EX collaborativo basato su *cloud*, utilizzato in alternativa a *Overleaf* per la compilazione di documenti di grandi dimensioni, come ad esempio l'**Analisi dei Requisiti**;
- *LanguageTool* - utilizzato per individuare errori ortografici e tipografici nei documenti prodotti
- *Script Python custom* - utilizzato per valutare la leggibilità dei documenti redatti in lingua italiana, calcolando l'indice di *Gulpease_G*

Gli altri strumenti trasversali utilizzati anche a supporto del processo di documentazione sono descritti nella sezione *4.2 Infrastruttura di Supporto*.

3.2 Gestione della configurazione

3.2.1 Scopo

Lo scopo di questa sezione è esplicitare le norme, le procedure e gli strumenti con cui il gruppo intende gestire la produzione e l'evoluzione di tutte le risorse (codice, documentazione e configurazioni) durante l'intero ciclo di vita del progetto **Code Guardian**.

3.2.2 Aspettative

Le attese che il gruppo **SkyNet** si pone con la gestione della configurazione sono:

- prevenire e semplificare l'individuazione di conflitti ed errori tra le diverse lavorazioni;
- uniformare gli strumenti e i flussi di lavoro utilizzati dai membri del gruppo;
- garantire la tracciabilità e la riproducibilità di ogni modifica, mantenendo sempre disponibile la cronologia e le versioni precedenti di ogni file.

3.2.3 Descrizione

Il processo di gestione della configurazione crea un ambiente organizzato e sistematico per la produzione di codice e documentazione. Tutti gli artefatti prodotti soggetti a modifica vengono identificati come *Elementi di Configurazione_G* e tracciati all'interno di un unico sistema di controllo versione centralizzato, in modo da rendere sempre ricostruibile la storia di ogni risorsa e riconducibile ogni modifica al suo autore.

3.2.4 Versionamento

Per garantire che la storia di ogni risorsa sia sempre ricostruibile, si adotta una convenzione di identificazione della versione basata sul formato a tre cifre:

$$[X].[Y].[Z]$$

dove:

- **X – Versione di produzione:** rappresenta la versione stabile associata ad approvazioni ufficiali e ai rilasci per le *milestone* principali (**RTB**, **PB**);
- **Y – Integrazione di piccoli incrementi:** indica l'aggiunta o l'integrazione di componenti e sezioni di rilevante entità (es. il completamento di una sotto-sezione o di una nuova funzionalità);
- **Z – Incremento di piccole dimensioni:** identifica modifiche minori, correzioni ortografiche o l'esito di attività di verifica; ogni incremento di questo tipo viene verificato dai **Verificatori** e approvato dal **Responsabile**.

Il numero di versione di ogni documento è riportato nella tabella delle informazioni principali e nel registro delle modifiche descritti nella sezione *3.1.2 Progettazione*. Lo stesso codice di versione è riportato anche nel nome dei documenti compilati pubblicati (ad esempio *norme_di_progetto_v0.8.1.pdf*), così da rendere immediatamente riconoscibile la versione di ciascun artefatto.

3.2.5 Sistemi Software Utilizzati

Per la gestione e il tracciamento delle differenti versioni degli artefatti, il gruppo adotta i seguenti strumenti:

- **Git:** sistema di controllo versione distribuito utilizzato per monitorare la cronologia delle modifiche locali e distribuite;
- **GitHub:** piattaforma di *hosting* usata per ospitare il *repository* remoto del progetto;
- **GitHub Projects e Issue:** utilizzati per la pianificazione e l'assegnazione dei compiti (*issue*) secondo la board *Scrumban*;
- **Pull Request:** meccanismo formale con cui le modifiche apportate nei *branch* di lavoro vengono revisionate da un **Verificatore** prima di essere integrate nel ramo principale;
- **GitHub Actions:** utilizzato per automatizzare il *deploy* della documentazione e del sito del gruppo su *GitHub Pages* ad ogni aggiornamento del ramo principale.

3.2.6 Struttura Repository

Repository utilizzato

Il *repository* ufficiale del gruppo è pubblico ed è raggiungibile al seguente indirizzo:

https://github.com/SkynetUniGroup/Code_Guardian

Organizzazione dei rami

L'organizzazione del *repository* si ispira al modello *Git Flow* ed è così articolata:

Rami Principali

- **main**: ramo contenente esclusivamente artefatti (codice e documentazione) approvati da almeno un **Verificatore** e dal **Responsabile**. Ogni stato del ramo **main** rappresenta una configurazione stabile del progetto.
- **develop**: ramo di sviluppo, contiene le *feature* completate e testate, confluite dai rami secondari.

Rami secondari

- **feature**
 - Nome: `feature/[ID_ISSUE]-[descrizione-breve-in-kebab-case]`.
 - Origine: **develop**.
 - Scopo: sviluppare e testare una nuova funzionalità isolatamente, senza interferire con il lavoro degli altri membri del team.
 - *Merge*: in **develop** tramite *Pull Request*.
 - Durata massima: 2 settimane. Se più lunga, suddividere in sotto-compiti.
- **release**
 - Nome: `release/[VERSIONE]`.
 - Origine: **develop**.
 - Scopo: preparazione del rilascio (test finali, bugfix minori...).
 - *Merge*: in **main** e **develop** tramite *Pull Request*.
 - Tag: creare un *tag* `v[VERSIONE]` su **main** dopo il *merge*.
- **hotfix**
 - Nome: `hotfix/[ID_ISSUE]-[descrizione-breve-sempre-in-kebab-case]`.
 - Origine: **main**.

- Scopo: correzione di bug critici in produzione.
- *Merge*: in `main` e `develop` tramite *Pull Request*.
- *Tag*: create un *tag* `v[VERSIONE]-hotfix[INCREMENTO]` su `main` dopo il *merge*.

Regole di *Commit*

Viene adottato lo standard *Conventional Commits*, che prescrive la seguente forma per i *commit*:

[tipo](obbiettivo): descrizione della modifica

dove il tipo è uno dei seguenti:

- **feat**: aggiunta di nuove funzionalità.
- **fix**: correzione di bug.
- **docs**: modifiche alla documentazione.
- **style**: modifiche alla formattazione o allo stile.
- **refactor**: modifiche alla struttura del codice che non cambiano il comportamento.
- **test**: aggiunta o modifica di test.
- **chore**: modifiche a configurazioni, dipendenze o *script* di *build*.

e l'obbiettivo è il modulo o il componente interessato.

Regole per le *Pull Request*

- Titolo: [Tipo] [Scope]: [Descrizione].
- Descrizione: *link* all'*issue*, motivazione, *screenshot* (se *UI*), lista delle modifiche.
- Revisione: almeno un **Verificatore** e il **Responsabile**.
- Controlli obbligatori:
 - Tutti i test superati.
 - Esame del *linter* passato.
 - Copertura dei test superiore alla soglia minima.
- *Merge*: *squash and merge* per rami `feature`, *merge commit* per `release/hotfix`.

Sincronizzazione

- `develop` deve essere sempre aggiornato con `main`.
- *Merge* regolare (settimanale) di `main` in `develop`.

- **Rami feature:** rebasing frequente su `develop` per evitare conflitti.

Organizzazione delle cartelle Nel ramo `main` gli artefatti sono organizzati nelle seguenti cartelle e file principali:

- **Documentazione/:** contiene tutta la documentazione del progetto, suddivisa in:
 - **LaTeX/:** sorgenti $\text{\LaTeX}$ dei documenti, con una sotto-cartella per ciascun documento (**Analisi dei Requisiti**, **Norme di Progetto**, **Piano di Progetto**, **Piano di Qualifica**, **Glossario** e, dal 21/8, i verbali) e la cartella **Template/** contenente i *template* documentali;
 - **Candidatura/:** documenti compilati (`.pdf`) relativi alla fase di candidatura al *capitolato* (valutazione dei capitolati, preventivo dei costi, lettera di presentazione);
 - **RTB/:** versioni ufficiali (`.pdf`) destinate alla revisione *Requirements and Technology Baseline*;
 - **PB/:** versioni ufficiali (`.pdf`) destinate alla revisione *Product Baseline*;
 - **Verbali/:** verbali degli incontri, distinti in **Interni/** ed **Esterni/**.
- **Src/:** contiene il codice sorgente dell'applicativo (agenti, *backend* e *frontend*) e i relativi test automatici, organizzato in due sottocartelle:
 - **PoC/:** contiene il codice sorgente del **Proof of Concept** nella sua seconda iterazione;
 - **MVP/:** ospita il codice sorgente del **Minimum Viable Product**.
- **Website/:** contiene i sorgenti del sito del gruppo, pubblicato tramite *GitHub Pages*;
- **Tools/:** contiene *script* e strumenti di generica utilità;
- **.github/workflows/:** contiene i *workflow* di *GitHub Actions* per l'automazione del *deploy* dei contenuti statici e la *CI/CD*;
- **.husky/:** contiene i file di configurazione degli *hook* di *Git*;
- **README.md** e **LICENSE:** rispettivamente la descrizione del *repository* e la licenza (*MIT*) con cui è distribuito il progetto;
- **.gitignore:** file configurato per escludere dal tracciamento i file di sistema generati automaticamente dai sistemi operativi (es. `.DS_Store`, `desktop.ini`).

3.2.7 Strumenti di Build

Gestione dipendenze

- *pnpm*: comandi `pnpm install`, `pnpm add -d [package]`, `pnpm remove -d [package]`.
 - Dipendenze: sono aggiunte **automaticamente** da `pnpm` a `package.json`, modificabile per configurazione e aggiunta di script. Soggetto a versionamento.
- *Lockfile*: `pnpm-lock.yaml` da "committare". **Mai** modificarlo manualmente.
- *Workspace*: struttura monorepo con `pnpm-workspace.yaml`.

Bundling e Dev Server

- *Vite*: configurazione in `vite.config.ts`.
 - Motori: *Oxc* (*transpiling*), *Rolldown* (*bundling*).
- *Script*: `pnpm dev` (avvio *dev server*), `pnpm build` (*build* produzione).
- *Output*: `/dist`. Escludere da *Git* via `.gitignore`.

Pipeline CI/CD

- Strumento: *GitHub Actions*. File di configurazione in `.github/workflows/`.
- *Workflow principali*
 1. Pull Request Check (`pr-check.yml`)
 - Trigger: *Pull Request* su qualsiasi *branch*.
 - Fasi:
 1. *Checkout* codice.
 2. Installazione di *pnpm* v11, *Node.js* v24 e dipendenze.
 3. Installazione di *Python*.
 4. Installazione della *cache* di *Python* *pip*.
 5. *Linting* *Typescript*.
 6. Test unitari *Typescript*.
 7. Installazione delle dipendenze di *Python* con *pip*.
 8. *Linting* *Python*.
 9. Test unitari *Python*.
 10. Test di integrazione.

- 11. *Build*.
 - Requisiti per il *merge*: tutte le fasi devono passare.
- 2. CI Staging (*ci-staging.yml*)
 - Trigger: *push* su *develop*.
 - Fasi: 1-10. come *Pull Request Check*.
 - Artefatti: salvati in */dist*.
- 3. Release Production (*release.yml*)
 - Trigger: *push tag v** su *main*.
 - Fasi:
 - 1-11. come *Pull Request Check*.
 - 12. Configurazione delle credenziali *AWS*.
 - 13. *Deploy frontend* su *Amazon S3*.
 - 14. Login *Amazon ECR*.
 - 15. *Build, tag, push* delle immagini *Docker*.
 - 16. *Deploy* su *Amazon ECS*.
 - Artefatti: build e tag di rilascio.
- Variabili d'ambiente
 - *NODE_ENV*: *test* per PR Check, *production* per Release.
 - Segreti: *GITHUB_TOKEN*, *AWS_ACCESS_KEY_ID*, *AWS_SECRET_ACCESS_KEY*.
- Artefatti
 - *Build* salvate per 7 gg in *GitHub Actions*.
 - *Log* conservati per 30 gg.

3.3 Verifica

Questo processo determina se gli artefatti del progetto soddisfino i requisiti e le condizioni stabiliti precedentemente. Ha inoltre lo scopo di controllare che le attività di sviluppo, documentazione e progettazione non introducano errori negli artefatti prodotti.

Per ogni modifica apportata alla documentazione, alla progettazione o al codice, la verifica non deve essere svolta dall'autore della modifica, ma da un altro membro del gruppo. Inoltre, per gli artefatti di progetto più rilevanti, possono essere richieste verifiche esterne da parte del **Proponente**.

Il processo di verifica ha due forme:

- **analisi statica**,
- **analisi dinamica**.

3.3.1 Verifica della documentazione

La verifica della documentazione controlla che i documenti siano sufficientemente leggibili, coerenti e completi, e che gli argomenti trattati siano affrontati correttamente e con un adeguato livello di approfondimento. La verifica quindi non si limita alla sola lettura delle modifiche apportate, né al controllo della loro leggibilità.

Le correzioni individuate vengono riportate evidenziando le parti da correggere nel file PDF, accompagnate da un breve commento.

3.3.2 Analisi statica

L'analisi statica non richiede l'esecuzione dell'oggetto di verifica e può quindi essere applicata in qualsiasi momento dello sviluppo del prodotto software. Può essere svolta su documentazione, progettazione, codice e qualsiasi prodotto di processo. Ha lo scopo di verificare la conformità ai requisiti e standard stabiliti, la presenza delle proprietà attese e l'assenza di difetti.

L'analisi statica può utilizzare metodi di lettura per prodotti semplici, oppure metodi formali basati sulla prova assistita di proprietà per prodotti più complessi.

I metodi di lettura, manuali o automatizzati, si distinguono principalmente per l'approccio iniziale adottato durante la verifica. Questi sono:

- *walkthrough* - il verificatore non parte da un'ipotesi precisa sui punti in cui è più probabile individuare difetti e quindi esamina l'artefatto in modo generale. Poiché i membri del gruppo hanno ancora poca esperienza, questo sarà il metodo adottato nella maggior parte delle verifiche semplici.
- *inspection* - il verificatore o lo strumento conosce quali difetti potrebbero essere presenti e in quali punti potrebbero essere localizzati.

L'efficacia di questi metodi dipende dall'esperienza del verificatore e dall'accuratezza degli strumenti utilizzati.

3.3.3 Analisi dinamica

L'analisi dinamica richiede l'esecuzione dell'oggetto di verifica ed è quindi applicabile al codice dopo che almeno una sua parte è stata implementata. Viene svolta tramite l'esecuzione di test, con l'obiettivo di verificare il comportamento del software e i risultati prodotti. L'analisi dinamica viene utilizzata anche nel processo di validazione.

I test utilizzati nell'analisi dinamica devono essere ripetibili, in modo da permettere una valutazione affidabile dei risultati nel tempo. Per ogni test è quindi necessario specificare:

- l'ambiente di esecuzione, comprendente hardware, software e stato iniziale
- le attese, cioè gli input accettabili e gli output attesi
- le procedure, cioè le modalità di esecuzione del test e di analisi dei risultati.

Il gruppo mira ad automatizzare il maggior numero possibile di test, così da potersi concentrare sull'analisi degli esiti, sull'individuazione e correzione degli errori e sulla loro esecuzione frequente, senza dover impiegare personale per ogni singola esecuzione.

3.3.4 Test di Unità

I test di unità servono a verificare il corretto funzionamento delle parti più piccole del software, chiamate unità, sottoponibili singolarmente al processo di verifica. Anch'essi fanno parte della progettazione dettagliata dell'architettura del software. Ogni test di unità viene implementato da un singolo **Programmatore**.

Framework

- *TypeScript*: *Vitest*. Configurazione in `vite.config.ts`.
- *Python*: *pytest*. Configurazione in `pytest.ini`.

Copertura Minima

- 75% globale.
- Misurata con *V8 (Vitest)* e `pytest-cov`.

Convenzioni

- Nome file di test:
 - *TypeScript*: `[nome_modulo].test.ts` o `[nome_modulo].spec.ts`.
 - *Python*: `test_[nome_modulo].py`.

- Nome dei test:
 - *TypeScript*: `describe("[modulo]", ()=>{it("[scenario]", ()=>{...}})`
 - *Python*: `test_[funzione]_[scenario]`.
- Struttura: *AAA pattern* (Arrange-Act-Assert).

Casi di test obbligatori

- Input validi e invalidi.
- Valori limite.
- Eccezioni attese.
- Casi limite specifici del dominio (es. token scaduto, formato non valido).

Mocking

- *TypeScript*: integrato in *Vitest* (es. `vi.fn()`, `vi.mock()`).
- *Python*: `unittest.mock`

Automazione

- Esecuzione con `pnpm test:unit` in *CI/CD* tramite *GitHub Actions* ad ogni *push* (*workflow* `deploy-staging.yml`) o *Pull Request* (*workflow* `pr-check.yml`) su `develop`.

3.3.5 Test di Integrazione

I test di integrazione verificano che le varie componenti del software, le tecnologie adottate e i servizi implementati comunichino, scambino dati e interagiscano correttamente. Servono quindi a testare funzionalità di livello più alto, che richiedono la collaborazione di più componenti distinti del software.

Framework *Vitest e pytest.*

Ambiente

- *Docker Compose* per servizi esterni (*MongoDB, API mock*).
- Variabili d'ambiente: prefisso `TEST_`.

Casi di test

- Interazione tra moduli.
- Flussi end-to-end (es. "Utente richiede audit→agente elabora→report salvato in DB").
- Integrazione con API esterne.

Dati di Test

- *Fixture*: file in `tests/fixtures/`.
- Pulizia: reset DB tra test (es. `pytest-mongodb fixture`).

Esecuzione

- Locale: `pnpm test:integration`
- *CI*: Esecuzione in *CI/CD* con *GitHub Actions* come parte dei workflow `pr-check.yml` (*Pull Request*) e `deploy-staging.yml` (*push* su `develop`), insieme ai test unitari.

3.4 Validazione

La validazione avviene nelle fasi finali del progetto e ha lo scopo di accertare che il prodotto realizzato sia conforme alle aspettative. In particolare, verifica che il prodotto software sia adeguato all'uso previsto e sia in grado di soddisfare i bisogni degli utenti finali.

Una corretta e continua applicazione del processo di verifica contribuisce al buon esito della validazione.

I risultati del processo di validazione verranno documentati nel **Piano di Qualifica**.

3.4.1 Test di Sistema

I test di sistema vengono eseguiti sul prodotto nel suo complesso, con l'obiettivo di verificare che le funzionalità implementate soddisfino i requisiti software definiti nell'**Analisi dei Requisiti**.

Dal punto di vista dell'utente finale, il prodotto viene testato nel maggior numero possibile di scenari d'uso, includendo anche i casi limite. Gli eventuali bug individuati vengono documentati e corretti. Eventuali bug non critici residui vengono documentati per il **Proponente**. È necessario che il prodotto soddisfi tutti i requisiti obbligatori stabiliti nell'**Analisi dei Requisiti**.

3.4.2 Test di Accettazione

Questo test segue i *test di sistema* e rappresenta il collaudo finale del prodotto. Viene svolto di fronte al **Committente**, durante il quale l'*MVP* viene dimostrato in modalità live. Il buon esito del collaudo consente il rilascio finale del prodotto e la conclusione del progetto.

4 Processi Organizzativi

4.1 Gestione di Processi

Questa sezione descrive come il gruppo gestisce le attività e i compiti previsti durante il progetto. Descrive inoltre i ruoli necessari per garantire che ogni obbligo venga assolto e la comunicazione e la coordinazione tra i membri del gruppo che rende possibili progressi notevoli.

4.1.1 Attività previste

Le attività previste per la gestione di processi sono le seguenti:

Inizializzazione - Le attività e i compiti da svolgere vengono identificati individuando gruppi di requisiti correlati, che possono essere obbligatori o facoltativi. È necessario valutare attentamente la fattibilità di ogni singola attività e compito analizzando la disponibilità delle risorse necessarie per lo svolgimento.

Pianificazione - È la pianificazione dell'esecuzione di attività e compiti. Il piano deve descrivere bene il compito, gli strumenti da utilizzare, *preventivo* e consuntivo dei costi, a chi viene assegnato il compito, scadenze e rischi potenziali.

Esecuzione e Controllo - Durante l'esecuzione del piano, per un'attività/compito è importante monitorare il progresso, i costi ed eventuali imprevisti. In caso di problemi il piano può essere modificato. Durante gli incontri con gli altri membri del gruppo, ed eventualmente anche con i tutor esterni, è necessario comunicare lo stato di avanzamento dell'attività.

Revisione e Valutazione - I risultati di attività/compiti devono essere valutati secondo i requisiti rilevanti. Lo stesso vale per i prodotti software.

Chiusura - Quando un'attività o un compito viene completato, ciò che viene prodotto va valutato rispetto alle aspettative del **Proponente**, in termini di qualità e completezza. Quindi si decide se l'attività può essere definita conclusa, o se è necessario continuare a lavorare su di essa.

4.1.2 Ruoli

Per assicurare che ogni aspetto di ogni attività precedente venga compiuto adottiamo sette ruoli. I ruoli hanno le loro responsabilità definite e vengono assegnati ai membri del gruppo. I membri si alternano nei vari ruoli, per motivi formativi.

Responsabile_G - Si occupa della comunicazione esterna e rappresenta il gruppo e il progetto. Gestisce le riunioni interne e redige i verbali. Prende le decisioni importanti nei casi in cui non si raggiunga un consenso all'interno del gruppo. In collaborazione con gli altri membri pianifica le attività e i compiti da svolgere e ne valuta il progresso tempestivamente. Mantiene aggiornato il documento **Piano di Progetto**. Si occupa dell'approvazione finale per la maggioranza dei compiti.

Amministratore_G - Definisce, controlla e mantiene l'ambiente *IT* di lavoro. Seleziona e mette in opera le risorse e gli strumenti informatici a supporto del *Way of Working*. Sceglie nuove tecnologie e modi di lavorare migliori e informa adeguatamente il gruppo per assicurare una transizione fluida. Redige il documento **Norme di Progetto**. Cerca di mantenere e migliorare il *Way of Working* del gruppo.

Analista - Ha il compito importante di stabilire e tracciare i requisiti del progetto. Deve informarsi sul dominio del problema e conoscerlo quanto possibile. Definisce gli *Use Case*. Il ruolo è maggiormente attivo nelle fasi iniziali di ogni *sprint* o iterazione.

Progettista - Determina le scelte realizzative per l'architettura di progetto intero e di componenti più piccoli. Deve informarsi sulle scelte architettureali, sul **system design_G** e le tecnologie che vengono utilizzate.

Programmatore - Contribuisce alla realizzazione e manutenzione del prodotto eseguendo per lo più le attività di codifica. Ha competenze tecniche e si attiene principalmente alle specifiche definite dall'**Analista** e alle scelte architettureali definite in fase di progettazione.

Verificatore_G - Verifica che il lavoro svolto soddisfi i requisiti e rispetti la qualità richiesta. Fornisce il riscontro e giudica se e dove sono necessari ulteriori interventi. Acquisisce informazioni adeguate sui vari argomenti per consentire una valutazione appropriata.

Uno stesso membro non può assumere il ruolo di **Verificatore** immediatamente dopo aver ricoperto il ruolo di **Analista**, **Progettista** o **Programmatore** perché comporterebbe la verifica del proprio lavoro.

4.1.3 Coordinamento

Riunioni Ci sono 3 tipologie di riunioni a cui è necessario partecipare: interne, esterne, **diari di bordo**.

Riunioni Interne

A queste riunioni partecipano esclusivamente i membri del gruppo **SkyNet** che lavorano sul progetto. Ogni membro è tenuto a partecipare.

Le riunioni si svolgono alla fine e all'inizio di ogni *sprint*, e in caso di necessità, qualora si verificano situazioni in cui sia necessario affrontare e risolvere un problema collettivamente.

Le tematiche discusse sono:

- Fare una *retrospettiva* dello *sprint* concluso e vedere come migliorare il *Way of Working* nello *sprint* successivo. Analizzare il grafico **Burndown_G** dello *sprint*.
- Fare un *Consuntivo* dei compiti e delle attività svolte e non durante lo *sprint*. Identificare le ragioni per cui alcuni compiti non sono stati compiuti e decidere come gestirli in futuro.
- Assegnare i ruoli per lo *sprint* successivo.

- Identificare le attività e i compiti su cui bisogna lavorare nello *sprint* successivo. Assegnare i compiti ai membri del gruppo. Ogni compito deve avere esattamente un assegnatario.
- Discutere eventuali rischi incontrati, come sono stati gestiti e come saranno gestiti se dovessero comparire in futuro.
- Discutere lo stato corrente della formazione del personale.
- Discutere altri argomenti importanti che il gruppo deve analizzare e prendere una decisione (ad es. se introdurre uno strumento nuovo per migliorare il *Way of Working*).

Per ogni riunione interna occorre redigere un verbale che riassume gli argomenti discussi e le decisioni prese durante l'incontro, in ordine cronologico.

Riunioni Esterne

Partecipano i membri del gruppo **SkyNet** che lavorano sul progetto e collaboratori dall'agenzia **Proponente**. Vengono svolte quando una delle parti ritiene che sia necessario incontrarsi. Le tematiche di solito sono:

- Avanzamento nello sviluppo del progetto.
- Lo stato corrente della formazione del personale del fornitore (gruppo **SkyNet**). Se e come il **Proponente** può fare il mentore e aiutare i membri di **SkyNet** a imparare certe tecnologie richieste.
- Chiarimenti dei requisiti da parte del fornitore.
- Altre tematiche di progetto che solleva il **Proponente**.
- *Demo* del prodotto, ovvero la presentazione degli incrementi software sviluppati durante lo *sprint*.
- Validazione, cioè l'accettazione formale dei requisiti implementati da parte del **Proponente**.

Per ogni riunione esterna occorre redigere un verbale che riassume gli argomenti discussi e le decisioni prese durante l'incontro, in ordine cronologico.

Diari di Bordo

I **diari di bordo** sono incontri periodici con il Prof. Vardanega, che rappresenta il **Committente**, e con gli altri gruppi del II lotto. Per ogni diario di bordo, il gruppo prepara una breve presentazione *PowerPoint* in cui riporta:

- le attività completate dalla riunione precedente
- le attività pianificate per il periodo successivo
- le difficoltà riscontrate

- eventuali dubbi su come procedere.

La presentazione viene esposta dal membro del gruppo che in quel periodo svolge il ruolo di **Responsabile**, con un tempo massimo di circa 4 minuti.

Lo scopo di questi incontri, svolti tramite *Zoom*, è monitorare l'andamento del progetto, confrontarsi sulle difficoltà incontrate e ricevere indicazioni dal Prof. Vardanega.

Comunicazioni

Le comunicazioni interne avvengono tramite:

- riunioni interne
- *Discord_G* - per messaggi raggruppati in base al tema in canali testuali appositi
- *WhatsApp_G* - per messaggi veloci e brevi

Le comunicazioni esterne avvengono tramite:

- la posta elettronica - per inviare email al **Proponente**, ai collaboratori dall'agenzia **Proponente** e ai **Committenti**
- *Slack_G* - per una comunicazione più rapida con Var Group
- riunioni esterne con il **Proponente**
- **diari di bordo** con i **Committenti**

4.2 Infrastruttura di Supporto

Questo processo si prende cura dell'infrastruttura necessaria per qualsiasi altro processo.

4.2.1 Attività previste

Le attività previste di questo processo sono:

Implementazione - definizione ed implementazione dell'infrastruttura adottata. È necessario considerare prestazioni, sicurezza, disponibilità, costi e vincoli di tempo.

Creazione - La configurazione dell'infrastruttura va pianificata e documentata.

Mantenimento - l'infrastruttura deve essere mantenuta, monitorata e modificata per assicurare il soddisfacimento dei requisiti che la necessitano.

Implementazione

Overleaf - Un editor $\text{\LaTeX}$ collaborativo basato su *cloud*. Utilizzato per la formattazione precisa dei file di documentazione e per il loro affinamento finale.

Google Docs - Un elaboratore di testi gratuito basato sul web. Utilizzato per la scrittura collaborativa dei file di documentazione.

Google Sheets_G - Un'applicazione per fogli di calcolo basata sul *cloud* che consente agli utenti di collaborare sui dati in tempo reale.

Gmail_G - Servizio di posta elettronica di *Google* basato sul web. Utilizzato principalmente per la comunicazione esterna.

Discord - Un'app di comunicazione che combina messaggistica istantanea, videochiamate e chat vocale in comunità organizzate per argomento, chiamate "server". Utilizzato per la comunicazione interna e come la piattaforma dove svolgiamo le nostre riunioni interne.

WhatsApp - Un'app di messaggistica istantanea, utilizzata per la comunicazione interna, veloce e breve.

GitHub - Una piattaforma basata sul *cloud* che funge da servizio di *hosting_G* per i *repository Git*, consentendo agli sviluppatori di archiviare, gestire, condividere e collaborare ai progetti di codice. È inoltre un sistema di *ticketing_G*.

Utilizzato per l'*hosting* del nostro *repository* di progetto e per tracciare ed organizzare i compiti (chiamati *issue_G*) pianificati.

Git - È un sistema di controllo di versione distribuito *open source_G*. Permette di gestire la cronologia delle modifiche ai file in modo non lineare, facilitando la collaborazione tra più persone attraverso la ramificazione (*branching_G*) e la fusione (*merging_G*) del codice, indipendentemente dalla piattaforma di *hosting* utilizzata.

GitHub Pages_G - È un servizio gratuito di *hosting* per siti statici che trasforma i *repository GitHub* direttamente in siti web ospitati. Lo utilizziamo per l'*hosting* del nostro sito web (https://skynetunigroup.github.io/Code_Guardian/) per la documentazione del progetto.

Creazione

Google Docs - È stato creato un file testuale condiviso per ciascun documento necessario per la documentazione di progetto completa: **Glossario, Norme di Progetto, Analisi dei Requisiti, Piano di Progetto, Piano di Qualifica** e altri documenti che saranno aggiunti successivamente.

Google Sheets - Sono stati creati due fogli di calcolo:

1. utilizzo risorse progetto - tracciamo quante risorse sono state consumate fino allo *sprint* corrente
2. grafico *Burndown* - tracciamo quante *issue* chiudiamo ad ogni giorno dello *sprint* e quante ne rimangono

Gmail - È stato creato un account di gruppo (skynetunigroup@gmail.com). Ogni membro può utilizzare liberamente l'account, previo consenso del gruppo per l'invio di email.

Discord - È stato creato un server di gruppo che si chiama "SkyNet - Code Guardian". All'interno ci sono molteplici canali testuali, divisi per argomento:

- Documentazione: doc-da-verificare, sviluppo-analisi-requisiti, sviluppo-piano-progetto, sviluppo-glossario, sviluppo-norme-progetto, sviluppo-piano-qualifica, verbali

- Coding: per ora vuoto
- Canali testuali singoli: generale, comunicazione-esterna, gestione-risorse

C'è anche un canale vocale chiamato Incontro Interno.

WhatsApp - È stato creato un gruppo *WhatsApp* chiamato “SkyNet SWE” con ogni singolo membro presente.

GitHub - Il nostro *repository* remoto “Code_Guardian” di progetto è presente al seguente indirizzo: https://github.com/SkynetUniGroup/Code_Guardian. Contiene il codice sorgente, la documentazione e le risorse per il nostro sito web.

Inoltre, è stato creato un “progetto” del *repository* chiamato “TO DO Blackboard”. Lo scopo è di implementare il nostro *Scrumban Way of Working*. Contiene sei colonne tra cui spostiamo tutte le *issue* aperte finché non vengono chiuse. Le colonne sono:

- Backlog - le *issue* su cui occorre iniziare a lavorare presto ma non è possibile iniziare ancora perché alcune altre devono essere completate per prime.
- Ready - le *issue* su cui è possibile iniziare a lavorare subito.
- In progress - le *issue* su cui si sta lavorando e progredendo.
- In review - le *issue* il cui assegnatario ha finito di lavorare e le vuole chiudere ma prima devono essere verificate. Al momento in cui una *issue* viene posta in questa colonna, spetta al **Verificatore** controllarne il risultato e fornire un riscontro.
- Done - le *issue* che sono state approvate per la chiusura da parte del **Verificatore**. Diventa il compito del **Responsabile** procedere con la revisione finale.
- Approved - le *issue* completate e verificate che possono essere chiuse.

GitHub Pages - È stato configurato un *workflow_G* di *Continuous Deployment_G* tramite *GitHub Actions_G*. Uno script *Python_G* processa i file sorgenti nel *repository* e ne automatizza la pubblicazione sulla pagina web del progetto, garantendo il costante allineamento tra documentazione e stato dell'ambiente online.

Overleaf, *Google Docs*, *Google Sheets* e *Git* non richiedevano una configurazione specifica.

4.3 Miglioramento di Processi

Il processo di miglioramento ha lo scopo di stabilire, valutare, misurare, controllare e migliorare i processi del ciclo di vita del software adottati dal gruppo.

4.3.1 Attività previste

Valutazione del processo - analisi dell'idoneità e dell'efficacia dei processi a intervalli appropriati per identificare i punti deboli da migliorare

Miglioramento del processo - identificazione e applicazione di modifiche migliorative a seguito della valutazione

Valutazione del processo Questa attività viene svolta a intervalli appropriati con lo scopo di identificare i punti di forza, criticità e le aree di miglioramento. È un'attività cruciale per assicurare l'utilizzo di processi davvero utili ed efficienti, permettendo di intervenire tempestivamente su quelli che risultano non sufficientemente buoni, evitando così lo spreco di risorse.

Al termine di ogni *sprint*, il gruppo svolge la *retrospettiva* esaminando i nuovi processi e i processi identificati come deboli. L'obiettivo è di documentare se e perché i risultati dei processi siano stati insoddisfacenti oppure se possono essere migliorati o velocizzati. Quest'attività viene svolta mettendo a confronto i risultati effettivi con quelli attesi attraverso una riflessione collettiva tra i membri coinvolti.

Miglioramento del processo A seguito della valutazione del processo per ogni punto debole si identificano i possibili miglioramenti scegliendo l'opzione più adatta. Dallo *sprint* successivo la modifica viene apportata e il processo viene osservato nuovamente. Questo permette di valutare se mantenere la modifica o abbandonarla, seguendo l'approccio del *Ciclo PDSA_G* di miglioramento.

4.4 Formazione del Personale

Questo processo è parte integrante di un progetto sfidante, che prevede l'uso di tecniche e tecnologie nuove. È volto a fornire e mantenere personale qualificato, allineando le competenze del gruppo alle esigenze tecniche del progetto.

4.4.1 Attività previste

Analisi delle competenze - valutazione delle conoscenze tecniche e tecnologiche di ogni membro, rilevanti per il progetto.

Implementazione del processo - identificazione delle competenze necessarie per lo svolgimento del progetto e sviluppo di un piano di formazione.

Implementazione del piano di formazione - ogni membro del gruppo è tenuto allo svolgimento tempestivo del proprio piano di formazione e a riferire periodicamente sull'avanzamento.

Analisi delle competenze Si confrontano le conoscenze di ogni membro del gruppo con le tecnologie indicate dal **Proponente** e con le altre competenze necessarie per il dominio del progetto. Ogni membro dichiara il proprio livello di conoscenza per ogni tecnologia. Lo scopo è di identificare le lacune principali affinché i membri abbiano un punto di partenza per l'apprendimento.

Implementazione del processo Viene effettuata una revisione dei requisiti del progetto per identificare più dettagliatamente le competenze tecniche e gestionali necessarie. In base alle competenze di ogni singolo membro, ai loro interessi e alle necessità del progetto, viene creato un piano di formazione personalizzato per ogni membro del gruppo. L'obiettivo non è che ogni membro sappia fare tutto, ma che le loro competenze individuali siano complementari e che la loro unione corrisponda all'insieme delle competenze

richieste dal progetto. Il piano specifica strumenti, tecnologie, livelli di approfondimento richiesti e scadenze.

5 Metriche per la Qualità

Il gruppo adotta un sistema di misurazione rigoroso e basato su metriche al fine di garantire un approccio quantitativo e oggettivo alla valutazione della qualità. L'obiettivo principale di questo sistema è fornire un quadro di riferimento strutturato per il monitoraggio costante delle attività, permettendo al gruppo di individuare tempestivamente eventuali criticità, prevenire l'accumulo di debito tecnico e perseguire un miglioramento continuo attraverso l'analisi dei dati raccolti durante l'intero ciclo di vita del progetto.

5.1 Classificazione, Nomenclatura e Struttura

Al fine di garantire l'univocità, la reperibilità e una chiara interpretazione dei dati, le metriche sono suddivise in due macro-categorie:

- **Metriche di Qualità di Processo:** indicatori utilizzati per monitorare l'andamento delle attività di progetto, l'efficienza del team e la gestione delle risorse (tempi, costi e stabilità dei requisiti). Regolano i processi primari, di supporto e organizzativi. Sono identificate dal prefisso **MP_[Progressivo]** (es. MP_01).
- **Metriche di Qualità di Prodotto:** indicatori focalizzati sulle caratteristiche degli artefatti prodotti (documentali o software). Sono identificate dal prefisso **MPD_[Progressivo]** (es. MPD_01).

Per definire ogni metrica in modo rigoroso, il gruppo ha stabilito un formato standardizzato per la loro tracciatura. Ogni metrica viene documentata definendo i seguenti campi:

- **Nome e Identificativo:** il codice univoco e la denominazione chiara della metrica.
- **Descrizione:** l'illustrazione dello scopo della misurazione e del contesto di applicazione.
- **Formula di calcolo:** il metodo matematico o la procedura per ottenere il valore quantitativo.
- **Valore di Accettazione:** la soglia minima necessaria per considerare il processo stabile o il requisito di prodotto soddisfatto. Risultati inferiori a tale soglia richiedono interventi correttivi immediati.
- **Valore Ottimale:** il livello target di eccellenza a cui il team aspira, che rappresenta lo standard qualitativo ideale per il progetto.

5.2 Obiettivi di Qualità di Prodotto

Per garantire che il sistema software finale sia robusto, affidabile e in linea con le aspettative del **Proponente**, il gruppo adotta il modello *ISO/IEC 9126_G*. Tale modello permette di quantificare le caratteristiche estrinseche ed intrinseche del software distribuito, declinandole nelle seguenti categorie e sotto-attributi.

5.2.1 Funzionalità

La funzionalità rappresenta la capacità del prodotto software di fornire funzioni in grado di soddisfare i requisiti, espliciti e impliciti, concordati in fase di analisi. Un'elevata funzionalità garantisce che il software faccia esattamente ciò per cui è stato progettato. I sotto-attributi misurati includono:

- **Adeguatezza:** la capacità di fornire un insieme di funzioni appropriate per i compiti specificati e per gli obiettivi dell'utente.
- **Interoperabilità:** l'abilità del sistema di interagire e scambiare informazioni correttamente con servizi esterni.
- **Conformità:** l'aderenza del software a standard, convenzioni o normative di dominio, incluse le specifiche contrattuali.

5.2.2 Affidabilità

L'affidabilità misura la stabilità operativa del prodotto nel tempo e in condizioni d'uso standard, valutando la sua capacità di mantenere inalterato il livello di prestazioni. Questo aspetto è critico per gestire, ad esempio, le comunicazioni con i modelli linguistici (*LLM_G*). L'affidabilità si declina in:

- **Maturità:** la capacità del sistema di prevenire malfunzionamenti derivanti da errori logici interni o da difetti del codice sorgente.
- **Tolleranza ai guasti (o agli errori):** la resilienza del sistema nell'assorbire comportamenti anomali, quali il superamento delle soglie dei *token_G* o le eccezioni a tempo di esecuzione durante il *parsing_G* del codice, garantendo il mantenimento delle operazioni fondamentali.

5.2.3 Usabilità

L'usabilità rappresenta la facilità con cui l'utente finale interagisce con il prodotto. Misura l'immediatezza dell'interazione, lo sforzo cognitivo richiesto e la capacità dell'interfaccia di prevenire errori umani. I fattori analizzati sono:

- **Comprensibilità:** la chiarezza con cui le funzionalità e lo scopo dell'interfaccia sono comunicati all'utente al primo utilizzo.
- **Apprendibilità (Learning Time):** il tempo e lo sforzo necessari a un nuovo utente per padroneggiare in autonomia le funzionalità principali, senza dover ricorrere a un esteso supporto manualistico.

5.2.4 Efficienza

L'efficienza definisce il rapporto ottimale tra il livello delle prestazioni fornite dal software e la quantità di risorse (sia computazionali che temporali) utilizzate. Viene valutata attraverso:

- **Comportamento temporale (Tempo di Risposta):** la velocità con cui l'interfaccia reagisce a un input dell'utente e il tempo medio di elaborazione impiegato dai singoli agenti operativi per completare le analisi in background.

5.2.5 Manutenibilità

La manutenibilità descrive lo sforzo richiesto per analizzare, isolare e implementare modifiche al software. Un software altamente manutenibile abbatta i costi futuri legati a *bug fix*, evoluzione e *refactoring*. Essa comprende:

- **Analizzabilità:** la facilità di diagnosticare la causa di un malfunzionamento e di identificare le parti di codice da correggere.
- **Modificabilità:** l'agilità nell'estendere l'applicativo, garantita da un basso accoppiamento.
- **Stabilità:** la resistenza del sistema all'introduzione di nuovi effetti collaterali indesiderati a seguito di una modifica al codice.
- **Testabilità:** lo sforzo necessario per validare l'efficacia delle modifiche attraverso metriche di copertura automatica e controllo della complessità ciclomatica.

5.2.6 Sicurezza

La sicurezza riveste un'importanza fondamentale in scenari in cui l'applicativo gestisce *codebase* esterne e credenziali. Valuta la capacità del sistema di blindare i dati e gli accessi, mediante:

- **Confidenzialità e Protezione Credenziali:** la garanzia che chiavi segrete, password, e *token API* non vengano esposti in chiaro nei *log*, nei report o nel codice sorgente tracciato nei sistemi di versionamento.
- **Integrità e Cifratura:** la conservazione sicura e l'*hashing* crittografico delle credenziali a riposo.
- **Validazione degli Input:** la corretta sanificazione dei dati in ingresso e degli *URL* forniti dall'utente, al fine di prevenire vulnerabilità strutturali e iniezioni di codice nocivo.

5.3 Metriche di Qualità di Processo

5.3.1 Processi primari

I processi primari regolano le attività direttamente collegate alla realizzazione materiale del prodotto software, dalla gestione della fornitura fino alle fasi di analisi, progettazione e codifica. L'obiettivo cardine è garantire la stabilità dei requisiti concordati con il **PropONENTE** ed evitare l'accumulo di debito tecnico o complessità strutturale eccessiva nelle fasi di implementazione dell'architettura e degli agenti.

Metrica	MP_1 - Earned Value (EV)
Formula	$EV = \frac{\text{Ore Completate Cumulative}}{\text{Ore Effettive Cumulative}} \times 100$
Utilità	Monitora l'efficienza organizzativa quantificando la percentuale di tempo pianificato (in proporzione alle task effettivamente chiuse) rispetto al tempo effettivo impiegato.

Metrica	MP_2 - Planned Value (PV)
Formula	$PV = \sum_{i=1}^n (\text{Ore Previste}_i \times \text{Tariffa Oraria}_i)$
Utilità	Rappresenta il costo totale preventivato e pianificato per le attività da svolgere nel periodo di riferimento.

Metrica	MP_3 - Actual Cost (AC)
Formula	$AC = \sum_{i=1}^n (\text{Ore Effettive}_i \times \text{Tariffa Oraria}_i)$
Utilità	Quantifica il costo effettivo e documentato sostenuto per le ore di lavoro realmente impiegate nel progetto.

Metrica	MP_4 - Estimated At Completion (EAC)
Formula	$EAC = \frac{BAC}{CPI}$ • BAC: Budget at Completion (totale previsto nel <i>preventivo</i>)
Utilità	Fornisce una stima aggiornata del costo totale finale del progetto proiettando l'efficienza di costo corrente (CPI).

Metrica	MP_5 - Estimate To Complete (ETC)
Formula	$ETC = EAC - AC_{cum}$
Utilità	Stima il budget residuo necessario per completare le attività del progetto a partire dallo stato attuale.

Metrica	MP_6 - Time Estimate At Completion (TEAC)
Formula	$TEAC = \frac{\text{Durata pianificata}}{SPI}$
Utilità	Proietta la durata complessiva finale del progetto basandosi sulla performance di avanzamento temporale finora registrata.

Metrica	MP_7 - Cost Performance Index (CPI)
Formula	$CPI = \frac{EV_{cum}}{AC_{cum}}$
Utilità	Misura l'efficienza economica del progetto indicando se il valore prodotto è in linea con i costi sostenuti.

Metrica	MP_8 - Schedule Performance Index (SPI)
Formula	$SPI = \frac{EV_{cum}}{PV_{cum}}$
Utilità	Misura l'efficienza temporale del progetto segnalando la presenza di eventuali anticipi o ritardi sulla pianificazione.

Metrica	MP_9 - Requirements Stability Index (RSI)
Formula	$RSI = \left(\frac{NR + NR_{modificati}}{NR} \right) \times 100$ • NR: Numero Requisiti
Utilità	Misura la maturità e la stabilità dei requisiti nel tempo, evidenziando l'entità delle variazioni.

5.3.2 Processi di supporto

I processi di supporto sono attività trasversali che sostengono i processi primari, focalizzandosi sulla corretta evoluzione e verifica della documentazione e del codice sorgente.

Metrica	MP_10 - Indice di <i>Gulpease</i>
Formula	$IG = 89 + \frac{300 \cdot NF - 10 \cdot NL}{NP}$ • NF: Numero Frasi • NL: Numero Lettere • NP: Numero Parole
Utilità	Valuta in modo oggettivo la complessità e il grado di leggibilità dei documenti testuali scritti in lingua italiana.

Metrica	MP_11 - Correttezza Ortografica
Formula	$CO = \text{Numero di Errori Ortografici Rilevati}$
Utilità	Quantifica gli errori grammaticali e di battitura al fine di garantire l'elevata qualità formale dei documenti rilasciati.

Metrica	MP_12 - Copertura del Codice (Code Coverage)
Formula	$CC = \frac{\text{Linee di Codice Eseguite dai Test}}{\text{Linee di Codice Totali}} \times 100$
Utilità	Misura la percentuale di codice sorgente effettivamente eseguita e coperta durante i test automatici per individuare carenze di verifica.

Metrica	MP_13 - Superamento dei Test (Test Success Rate)
Formula	$TSR = \frac{\text{Test Superati}}{\text{Test Eseguiti}} \times 100$
Utilità	Indica la percentuale di test automatici con esito positivo per certificare la stabilità del software e l'assenza di regressioni.

5.3.3 Processi organizzativi

I processi organizzativi sovrintendono all'efficienza metodologica del team, monitorando anomalie di pianificazione temporale (ritardi) e sforamenti dei costi preventivati.

Metrica	MP_14 - Rischi non Preventivati
Formula	$PRI = \frac{N_{rischi\ inattesi}}{N_{rischi\ totali\ manifestati}} \times 100\%$
Utilità	Valuta la qualità dell'analisi preventiva dei rischi misurando la percentuale di criticità verificatesi ma non identificate nel registro.

Metrica	MP_15 - Time Efficiency (TE)
Formula	$TE = \frac{\text{Ore Completate Cumulative}}{\text{Ore Effettive Cumulative}} \times 100$
Utilità	Monitora l'efficienza organizzativa quantificando la percentuale di tempo pianificato (in proporzione alle task effettivamente chiuse) rispetto al tempo effettivo impiegato.

5.4 Metriche di Qualità di Prodotto

Per garantire la qualità del sistema software, il gruppo adotta lo standard *ISO/IEC 9126*. Tale modello permette di quantificare le caratteristiche estrinseche ed intrinseche del software distribuito.

5.4.1 Funzionalità

La funzionalità rappresenta la capacità del prodotto software di fornire funzioni in grado di soddisfare i requisiti espressi e impliciti concordati con il **Proponente**.

Metrica	MPD_1 - Copertura dei Requisiti Obbligatori
Formula	$CRO = \frac{N_{ROS}}{N_{ROT}} \times 100$ • N_{ROS}: Numero Requisiti Obbligatori Soddisfatti • N_{ROT}: Numero Requisiti Obbligatori Totali
Utilità	Percentuale dei requisiti funzionali obbligatori soddisfatti dall'insieme degli agenti.

Metrica	MPD_2 - Copertura dei Requisiti Desiderabili
Formula	$CRD = \frac{N_{RDS}}{N_{RDT}} \times 100$ • N_{RDS}: Numero Requisiti Desiderabili Soddisfatti • N_{RDT}: Numero Requisiti Desiderabili Totali
Utilità	Percentuale dei requisiti funzionali classificati come desiderabili effettivamente soddisfatti dal prodotto finale.

Metrica	MPD_3 - Copertura dei Requisiti Opzionali
Formula	$CROP = \frac{N_{ROS}}{N_{ROT}} \times 100$ • N_{ROS}: Numero Requisiti Opzionali Soddisfatti • N_{ROT}: Numero Requisiti Opzionali Totali
Utilità	Percentuale dei requisiti funzionali classificati come opzionali effettivamente soddisfatti dal prodotto finale.

5.4.2 Affidabilità

L'affidabilità misura la capacità del sistema di mantenere costanti le proprie prestazioni nel tempo, minimizzando i fallimenti applicativi dovuti a downtime delle *API* esterne (*LLM*) o a eccezioni di *runtime_G* nel *parsing* delle *codebase*.

Metrica	MPD_4 - Densità dei Guasti (Anomalie Codice)
Formula	$DG = \frac{NF}{KLOC}$ • NF: Numero di failure rilevati • $KLOC$: Migliaia di linee di codice
Utilità	Indica la percentuale di operazioni fallite sul totale delle richieste eseguite in base alla mole del codice sorgente.

Metrica	MPD_5 - Rate Limiting
Formula	$RL = \frac{NEGS}{NET} \times 100$ • <i>NEGS</i>: Numero di eccezioni di Rate Limiting gestite con successo • <i>NET</i>: Numero di eccezioni di Rate Limiting totali generate
Utilità	Percentuale di successo nella gestione delle eccezioni di superamento soglia dei <i>token API</i> esterni.

Metrica	MPD_6 - Accuratezza delle Risposte AI
Formula	$ARA = \frac{NRV}{NRT} \times 100$ • <i>NRV</i>: Numero di report AI validate come corretti • <i>NRT</i>: Numero di report AI totali generati
Utilità	Percentuale di report validati come corretti e privi di <i>allucinazioni_G</i> (AI) evidenti.

Metrica	MPD_7 - <i>Indice Flesch Reading Ease_G</i>
Formula	$FRE = 206.835 - 1.015 \cdot \left(\frac{NP}{NF}\right) - 84.6 \cdot \left(\frac{NS}{NP}\right)$ • <i>NP</i>: Numero Parole • <i>NF</i>: Numero Frasi • <i>NS</i>: Numero Sillabe
Utilità	Calcola il grado di leggibilità dei report di Business generati in lingua inglese dall'Agente <i>Changelog_G</i> .

Metrica	MPD_8 - Branch Coverage
Formula	$BC = \frac{NRDE}{NRDT} \times 100$ • <i>NRDE</i>: Numero di Rami Decisionali Eseguiti dai test • <i>NRDT</i>: Numero di Rami Decisionali Totali
Utilità	Percentuale di rami logico-decisionali del codice sorgente che vengono attraversati e verificati durante l'esecuzione di test automatici.

5.4.3 Usabilità

L'usabilità rappresenta la capacità del prodotto software di essere compreso, appreso e utilizzato con facilità dall'utente finale. Essa misura l'immediatezza dell'interazione con il sistema, minimizzando il rischio di errori operativi e lo sforzo cognitivo richiesto per sfruttare appieno le funzionalità offerte.

Metrica	MPD_9 - Tempo di Apprendimento (Learning Time)
Formula	$LT = \frac{\sum_{i=1}^n T_{apprendimento.i}}{n}$ • n: Numero di utenti coinvolti nella sessione di test • $T_{apprendimento.i}$: Tempo impiegato dall'utente i-esimo per completare le funzionalità senza assistenza
Utilità	Tempo massimo necessario affinché un nuovo utente riesca a comprendere e utilizzare autonomamente le funzionalità principali del sistema, senza ricorrere a manuali o supporto esterno.

Metrica	MPD_10 - Tasso di Errore (Error Rate)
Formula	$ER = \frac{N_E}{N_{AT}} \times 100$ • N_E: Numero di errori (azioni errate o non valide) commessi • N_{AT}: Numero totale di azioni eseguite per lo scenario d'uso
Utilità	Percentuale di azioni errate, non valide o annullate commesse dall'utente durante la normale interazione con l'applicativo per completare uno specifico scenario d'uso.

5.4.4 Efficienza

L'efficienza definisce la relazione tra il livello di prestazioni del software e la quantità di risorse utilizzate.

Metrica	MPD_11 - Tempo di Risposta Interfaccia
Formula	$TRI = \frac{\sum_{i=1}^n (T_{risposta.i} - T_{richiesta.i})}{n}$ • n: Numero di misurazioni effettuate • $T_{risposta.i}$: Istante in cui il sistema fornisce il feedback visivo per la richiesta i-esima • $T_{richiesta.i}$: Istante in cui l'utente invia l'input o l'azione per la richiesta i-esima
Utilità	Tempo medio di attesa tra un'azione dell'utente nell'interfaccia grafica e il feedback visivo del sistema, escludendo l'esecuzione in background degli agenti.

Metrica	MPD_12 - Tempo di Risposta dell'Agente
Formula	$TRA = \frac{\sum_{i=1}^n (T_{completamento_analisi_i} - T_{avvio_agente_i})}{n}$ • n: Numero di esecuzioni dell'agente misurate • $T_{completamento_analisi_i}$: Istante in cui l'agente restituisce l'output elaborato per l'esecuzione i-esima • $T_{avvio_agente_i}$: Istante in cui la richiesta viene instradata all'agente (LLM) per l'esecuzione i-esima
Utilità	Tempo medio impiegato da un singolo agente per elaborare i dati, comunicare con i servizi esterni ed emettere l'output definitivo.

Metrica	MPD_13 - Tempo di Autenticazione
Formula	$TA = \frac{\sum_{i=1}^n (T_{successo_i} - T_{invio_credenziali_i})}{n}$ • n: Numero di tentativi di autenticazione misurati • $T_{successo_i}$: Istante in cui il sistema sblocca i permessi e la dashboard per il tentativo i-esimo • $T_{invio_credenziali_i}$: Istante in cui l'utente richiede l'accesso tramite credenziali per il tentativo i-esimo
Utilità	Tempo medio impiegato dal sistema per completare la verifica delle credenziali nel <i>database_G</i> , validare il <i>token</i> e autorizzare l'accesso utente.

5.4.5 Manutenibilità

La manutenibilità descrive lo sforzo richiesto per eseguire modifiche al software, inclusi correzioni, miglioramenti o adattamenti degli agenti a nuovi modelli linguistici.

Metrica	MPD_14 - <i>Prompt_G</i> Isolation (Disaccoppiamento dei <i>Prompt</i>)
Formula	$PI = \frac{N_{PI}}{N_{PT}} \times 100$ • N_{PI}: Numero di <i>prompt</i> archiviati in file di configurazione isolati • N_{PT}: Numero di <i>prompt</i> totali utilizzati nel sistema
Utilità	Percentuale di <i>prompt</i> isolati rispetto alla logica di <i>backend_G</i> , al fine di garantire una facile modificabilità senza alterare il codice sorgente.

Metrica	MPD_15 - <i>Complessità Ciclomatica_G</i>
Formula	$CC = E - N + 2P$ • E: Numero di archi nel grafo di controllo del flusso • N: Numero di nodi (istruzioni) nel grafo • P: Numero di componenti connesse
Utilità	Misura il grado di complessità logica dei metodi e delle funzioni del codice sorgente calcolando i percorsi linearmente indipendenti.

Metrica	MPD_16 - <i>Code Smell</i>
Formula	$CS = N_{CS}$ • N_{CS}: Numero totale di <i>code smell_G</i> rilevati tramite <i>analisi statica</i>
Utilità	Indica il numero di difetti di progettazione o implementazione nel codice (es. codice morto, importazioni superflue), che ne riducono la manutenibilità.

Metrica	MPD_17 - Duplicazione del Codice
Formula	$DC = \frac{L_{CD}}{L_{CT}} \times 100$ • L_{CD}: Linee di codice sorgente duplicate • L_{CT}: Linee di codice sorgente totali
Utilità	Percentuale di linee di codice sorgente duplicate all'interno del progetto, indicatore di una scarsa modularità.

5.4.6 Sicurezza

Le metriche di sicurezza valutano la capacità del sistema di proteggere i dati sensibili degli utenti, le credenziali di accesso ai servizi esterni e il codice sorgente analizzato.

Metrica	MPD_18 - Esposizione di Credenziali
Formula	$EC = N_{CE}$ • N_{CE}: Numero totale di chiavi, password o <i>token API</i> trovati esposti in chiaro
Utilità	Numero di <i>token API</i> , password o chiavi segrete esposte in chiaro nel codice sorgente, nei <i>log</i> di sistema o nei report generati.

Metrica	MPD_19 - Cifratura dei Dati Sensibili
Formula	$CDS = \frac{N_{DSC}}{N_{DST}} \times 100$ • N_{DSC}: Numero di record di dati sensibili archiviati in formato cifrato o sottoposti ad <i>hashing</i> • N_{DST}: Numero totale di record di dati sensibili memorizzati nel <i>database</i>
Utilità	Percentuale di password degli utenti e <i>token</i> di accesso esterni memorizzati in modo sicuro utilizzando algoritmi crittografici.

Metrica	MPD_20 - Input Validation
Formula	$IV = \frac{N_{IV}}{N_{IT}} \times 100$ • N_{IV}: Numero di input sanificati e validati con successo • N_{IT}: Numero totale di input forniti dall'utente e intercettati dal sistema
Utilità	Percentuale di input forniti dall'utente (inclusi <i>URL</i> di <i>repository</i> , <i>token</i> e configurazioni) sottoposti a validazione prima dell'elaborazione per prevenire iniezioni.